



RAT — Remote Access Topologie

Projektdokumentation

Schaffer Christof · Reichart Tobias

WPF-Client (.NET 9) · FastAPI-Backend · REST + JWT

Projektzeitraum: **21.05.2026 – 18.06.2026**

📦 **Repository (Client):** <<https://github.com/sumpfel/RAT-Client>>

📦 **Repository (Backend):** <<https://github.com/sumpfel/RAT-Backend>>

Inhaltsverzeichnis

1. Projektbeschreibung
 2. Projektplanung (Lastenheft)
 3. Umsetzungsdetails (Pflichtenheft)
 4. Softwarevoraussetzungen
 5. Funktionsblöcke bzw. Architektur
 6. Detaillierte Beschreibung der Umsetzung
 7. Mögliche Probleme und ihre Lösung
 8. Projekttagebuch
 9. Einsatz von KI
 10. Quellen für Bilder und Medien
-

1. Projektbeschreibung

Unser Projekt heißt **Remote Access Topologie (RAT)**. Es ist eine Software, mit der man zentral sein eigenes **reales Netzwerk** überwachen, überprüfen und konfigurieren kann (z. B. über SNMP). Die Oberfläche orientiert sich an **Cisco Packet Tracer**: Geräte werden als Knoten auf einer Topologie-Leinwand dargestellt und mit Kabeln verbunden.

Ursprünglich war geplant, dass man per GUI auch Ports und die komplette Gerätekonfiguration vornehmen kann. Da der Projektumfang dafür im gegebenen Zeitrahmen zu groß gewesen wäre, konzentrieren wir uns auf:

- **Automatische Verbindung** zu Geräten per **SSH, Telnet, SFTP und SCP** auf Mausklick,
- **SNMP-Funktionen** (Get / Set / Walk) inklusive MIB-Browser,
- **automatische Netzwerk-Erkennung** (Discovery) mit **nmap** und **tracert**,
- eine **zentrale, mehrbenutzerfähige Datenhaltung** über ein FastAPI-Backend mit

Rechte-Modell.

RAT besteht aus zwei Repositories: dem **C#/WPF-Client** (dieses Repository) und dem

FastAPI-Backend (Python), das die Topologie samt Geräten, Interfaces, Verbindungen,

Logins und Berechtigungen persistiert.

2. Projektplanung (Lastenheft)

Das Lastenheft beschreibt, **was** das System aus Sicht der Anwender leisten soll.

2.1 Zielsetzung

Eine Desktop-Anwendung, die ein reales Netzwerk grafisch abbildet und es erlaubt, Geräte zentral zu inventarisieren, zu erreichen und auszulesen — ohne sich an jedem Gerät einzeln anmelden zu müssen.

2.2 Anforderungen (Must-have / Nice-to-have)

ID	Anforderung	Beschreibung	Priorität	Implementiert
MH1	Drag & Drop Devices	Netzwerkgeräte wie z. B. Router sollen per Drag and Drop bewegt werden können und auf den Canvas gezogen werden können.	Muss	[x] Ja / [] Nein
MH2	Canvas mit Geräten und Verbindungen	Es soll einen Canvas geben, auf dem die Geräte und Verbindungen im Netzwerk dargestellt werden.	Muss	[x] Ja / [] Nein
MH3	Mehrere User	Es soll mehrere User geben mit unterschiedlichen Rechten.	Muss	[x] Ja / [] Nein
MH4	SSH/Telnet zu den Clients	SSH- oder Telnet-Verbindungen zu den Clients per Knopfdruck.	Muss	[x] Ja / [] Nein
NH1	Ratten-Icons	Eigene Ratten-Themed Icons anstatt der Standard-Windows-Versionen.	Kann	[x] Ja / [] Nein
NH2	SNMP	Clientgeräte können per SNMP angesehen werden.	Kann	[x] Ja / [] Nein
NH3	Benutzerrechte für Geräte	User sollen unterschiedlich mit Netzwerkgeräten interagieren können, je nachdem wie diese konfiguriert wurden.	Kann	[x] Ja / [] Nein
NH4	Dark / Lightmode	Option zum Umschalten der Benutzeroberfläche zwischen einem dunklen und einem hellen Design.	Kann	[x] Ja / [] Nein
NH5	Toggle für Interface-Namen	Ein-/Ausschalter (Toggle), um die Namen von Schnittstellen in der UI ein- oder auszublenden.	Kann	[x] Ja / [] Nein
NH6	Einstellbarer Zoom / UI-Größe	Möglichkeit für den Nutzer, die Skalierung der Benutzeroberfläche bzw. den Zoomfaktor anzupassen.	Kann	[x] Ja / [] Nein

Über diese Anforderungen hinaus implementiert (Bonus): automatische Netzwerk-Erkennung

(nmap + tracer), SFTP/SCP-Dateitransfer, MIB-Browser, persistente Backend-Speicherung,

lokaler Offline-Modus, Erststart-Setup mit nmap-Installation und Desktop-Verknüpfung.

2.3 Nicht-funktionale Anforderungen

- **Bedienbarkeit:** an Cisco Packet Tracer angelehnte, intuitive Topologie-Ansicht;

einheitliches, „ratiges“ braunes Theme (Light/Dark) mit eigenen Vektor-Icons.

- **Barrierefreiheit der Ansicht:** app-weiter Zoom (50 %–300 %).
- **Robustheit:** Fehler beim Backend/Netzwerk dürfen die Anwendung nicht abstürzen lassen.
- **Sicherheit:** Authentifizierung per JWT, Passworrichtlinie, abgestufte Rechte.
- **Wartbarkeit:** saubere Schichtentrennung (Logik / Daten / UI) und MVVM in der UI.

2.4 Zuständigkeiten (laut Planung)

- **Tobias:** Schicht 3 (UI/Präsentation) und Teile von Schicht 1.
- **Christof:** Schicht 2 (`NetworkObjectGraph` und alle Unterklassen) sowie kleine Teile von

Schicht 1.

- **Kooperation:** `MainController`.

2.5 Abgrenzung

Eine vollständige GUI-gestützte Port-/Gerätekonfiguration (wie in Packet Tracer) ist **nicht** Teil dieses Projekts. Der Fokus liegt auf Erreichbarkeit (SSH/Telnet/FTP), SNMP und der automatischen Topologie-Erkennung.

3. Umsetzungsdetails (Pflichtenheft)

Das Pflichtenheft beschreibt, **wie** die Anforderungen technisch umgesetzt werden.

3.1 Technische Umsetzung der Anforderungen

Anforderung	Umsetzung
F1 / F2	<code>NetworkObject</code> (Logik) + <code>NetworkObjectViewModel</code> + <code>NetworkObjectView</code> ; Seitenleiste <code>NetworkObjectListView</code> . Knotenposition (X , Y) wird im Modell gehalten und per Binding auf einem <code>Canvas</code> gezeichnet.
F3	<code>NetworkConnection</code> (Typ <code>Wired / Wireless</code> , Speed, Name, Note) + <code>NetworkConnectionViewModel</code> ; Kabel werden als <code>Line</code> gezeichnet, WLAN gestrichelt. Bearbeiten über <code>EditConnectionWindow</code> .
F4	<code>NetworkObjectSettingsWindow</code> mit Tabs für Specs, Interfaces (<code>UpdateInterfaceWindow</code> , <code>SelectInterfaceWindow</code>), Logins (<code>UpdateLoginWindow</code>), SNMP und Access Control.
F5	<code>NetworkObject</code> kapselt SSH.NET (<code>SshClient</code> / <code>SftpClient</code> / <code>ScpClient</code>) und ein Raw-TCP-Telnet. <code>EnsureConnected()</code> öffnet automatisch die passende Sitzung, wenn ein Kabel vom eigenen PC und ein passender Login existieren.
F6	<code>NetworkObject.GetSnmp/SetSnmp/WalkSnmp</code> über Lexm.SharpSnmpLib ; <code>MibCatalog</code> liefert gängige OIDs für den Browser.
F7	<code>Discovery/NmapService</code> ruft <code>nmap</code> (Ping-Sweep <code>-sn</code> , Port-Scan <code>-F</code> , OS-Erkennung <code>-O</code>) und <code>tracert</code> auf; <code>DeviceClassifier</code> rät den Gerätetyp aus offenen Ports.
F8	Anmeldung über <code>LoginView</code> → <code>DatabaseConnection.Login()</code> (OAuth2/JWT). Rechte-Modell <code>AccesRights</code> (Hidden/See/Edit/Admin/Owner) in <code>NetworkObject</code> .
F9	<code>RAT_Data.DatabaseConnection</code> (<code>IDatabaseConnection</code>) spricht das FastAPI-Backend per HTTP/JSON an.
F10	<code>DatabaseConnectionMock</code> als funktionierende, speicherinterne No-Op-Implementierung; „Use locally only“ auf dem Login-Screen.

3.2 Datenmodell (Kern)

- `NetworkObjectGraph` — die gesamte Topologie (Liste von `NetworkObject`).
- `NetworkObject` — ein Gerät: Typ, Name, Position, Specs (OS/CPU/GPU/RAM), Interfaces,

Settings (Logins, SNMP), Zugriffsrechte.

- `NetworkObjectInterface` — Netzwerkschnittstelle: Name, Speed, IP, offene Ports,

zugehörige Verbindung.

- `NetworkConnection` — Kabel/Funkstrecke zwischen zwei Interfaces.
- `Login` / `SnmpSettings` — Zugangsdaten pro Gerät.
- `AccessRight` / `NetworkUser` — Rechte pro Benutzer und Gerät.

3.3 Qualitätssicherung

- Unit-Tests** mit **xUnit** (`RAT_Tests`) für die Logikschicht (IP-/Port-Validierung,

`DeviceDescriptor`-Hierarchie, Rechte-Modell, `DeviceClassifier`).

- Rolling-File-Logging** (`Logging/AppLogger`): pro Programmstart eine neue Logdatei unter

`logs/`, es werden die neuesten drei behalten; unbehandelte UI-Exceptions werden geloggt.

- API-Dokumentation** des Backends in `API_documentation.md`.

4. Softwarevoraussetzungen

4.1 Laufzeit / Build

Komponente	Version
.NET SDK	9.0
Ziel-Framework (UI)	net9.0-windows (WPF, UseWPF)
Ziel-Framework (Logik/Daten/Tests)	net9.0
Betriebssystem	Windows 10/11 (WPF + System.Management /WMI sind Windows-spezifisch)
IDE (empfohlen)	Visual Studio 2022 / 2026

4.2 NuGet-Pakete

Paket	Version	Projekt	Zweck
Lextm.SharpSnmpLib	12.5.7	RAT_Logik	SNMP Get/Set/Walk
SSH.NET	2025.1.0	RAT_Logik	SSH-Shell, SFTP, SCP
System.Management	10.0.8	RAT_Logik	Geräte-Specs/NICs via WMI
xunit	2.9.2	RAT_Tests	Unit-Test-Framework
xunit.runner.visualstudio	2.8.2	RAT_Tests	Test-Runner
Microsoft.NET.Test.Sdk	17.12.0	RAT_Tests	Test-Infrastruktur
coverlet.collector	6.0.2	RAT_Tests	Code-Coverage

4.3 Externe Werkzeuge (zur Laufzeit, optional)

Werkzeug	Verwendung
nmap (Installer nmap-7.99-setup.exe)	Netzwerk-Discovery (Ping-Sweep, Port-/OS-Scan). Wird beim Erststart optional automatisch installiert; ohne nmap ist nur der „Discover“-Button deaktiviert.
tracert (Windows-Bordmittel)	Ermittlung des ersten Hops (Router) und der Internet-Erreichbarkeit.

4.4 Backend (separates Repository)

- Python 3 mit FastAPI, SQLAlchemy, OAuth2-Passwd-Flow / JWT.
- Standard-Basis-URL im Client: http://127.0.0.1:8000.
- Details siehe API_documentation.md.

5. Funktionsblöcke bzw. Architektur

Die Anwendung ist in **drei Schichten** plus eine Test-Schicht aufgeteilt (Visual-Studio-Projekte). Die Abhängigkeiten verlaufen strikt von oben nach unten — die Logik kennt weder Daten- noch UI-Schicht.





5.1 RAT_Logik — Logikschicht (Schicht 2)

Domänenmodell und Geräteinteraktion, **ohne** UI- oder Datenbankabhängigkeit:

- **Topologie:** `NetworkObjectGraph`, `NetworkObject`, `NetworkObjectInterface`,

`NetworkConnection`, `IP`, `ConnectionSpeed`.

- **Geräte-Typisierung:** `DeviceDescriptor` (abstrakte Hierarchie: PC/Router/Switch/Server/

Client/Hub/Cloud/AccessPoint → Icon + Label), `DeviceClassifier` (Typ aus offenen Ports).

- **Erreichbarkeit:** SSH/SFTP/SCP (SSH.NET) und Telnet in `NetworkObject`; `EnsureConnected`.
- **SNMP:** `GetSnmp/SetSnmp/WalkSnmp`, `SnmpSettings`, `MibCatalog`, `PortNames`.
- **Sicherheit/Benutzer:** `AccessRight`, `Permission` / `NetworkUser`, `PasswordPolicy`,

`Session`, `Login`.

- **Hostinfos:** `HostInterfaceInfo`, WMI-Auslesen der eigenen Specs.

5.2 RAT_Data — Datenschicht (Teil von Schicht 1)

- `IDatabaseConnection` — Schnittstelle für alle CRUD-Operationen.
- `DatabaseConnection` — reale HTTP/JSON-Anbindung an das Backend inkl. **automatischer

JWT-Erneuerung**.

- `DatabaseConnectionMock` — Offline-No-Op-Implementierung für den lokalen Modus.
- DTOs: `Account`, `User`, `UserSettings`.

5.3 RAT_WPF — Präsentationsschicht (Schicht 3, MVVM)

- **Views:** `TopologyView`, `LoginView`, `NetworkObjectView`, Settings-/Dialog-Fenster.
- **ViewModels:** `TopologyViewModel`, `LoginViewModel`, `NetworkObjectViewModel`,

`NetworkConnectionViewModel`, `SettingsViewModel` ...

- **Commands:** `LoginCommand`, `LogoutCommand`, `NetworkObjectAddConnectionCommand`,

`NetworkObjectDeleteCommand` ...

- **Stores:** `NavigationStore`, `DatabaseConnectionStore` (app-weite Verbindung).
- **Querschnitt:** `Themes` (Light/Dark, Icons, Zoom, DisplaySettings), `Discovery`

(`NmapService`), `Setup` (Erststart), `Logging` (`AppLogger`), `Converters`.

6. Detaillierte Beschreibung der Umsetzung

6.1 Topologie-Leinwand und MVVM

Die Topologie wird über zwei übereinanderliegende `ItemsControl` auf einem großen `Canvas` (6000 × 4000) gezeichnet: unten die **Geräte** (Drag & Drop), darüber die **Verbindungen** (klickbare Kabel-Trefferlinien). Die Leinwand liegt in einem `ScrollView` und lässt sich mit der Maus verschieben (Drag-to-Pan) sowie über Scrollbalken navigieren. Geräteknoten binden ihre Position (X/Y) per `TwoWay`-Binding; Verbindungen aktualisieren sich automatisch, wenn sich ein Endknoten bewegt. Knotenpositionen werden auf die Leinwand geklemmt, damit nichts in unerreichbarem Bereich landet.

6.2 Geräte-Erkennung (Discovery)

`NmapService` ermittelt zuerst per **Ping-Sweep** (`nmap -sn`) schnell alle Hosts und zeichnet die Topologie sofort. Anschließend werden **pro Gerät** im Hintergrund die offenen Ports (`nmap -F`, schnell, ohne Admin-Rechte) und optional das Betriebssystem (`nmap -O`, langsam, benötigt Adminrechte) ermittelt — die Ports erscheinen am Knoten, sobald der schnelle Scan fertig ist, ohne auf den OS-Scan zu warten. `tracert` identifiziert den **Router** (erster Hop) und ob das **Internet** erreichbar ist (dann wird eine **Cloud** hinter dem Router ergänzt). `DeviceClassifier` rät den Gerätetyp aus den offenen Ports (z. B. Datenbank- oder mehrere Dienst-Ports ⇒ Server; nur SSH oder nur HTTP ⇒ vermutlich Switch). Erkannte Geräte werden im Kreis um einen Switch angeordnet; bei WLAN-Anbindung wird zusätzlich ein **Access Point** zwischen PC und Switch modelliert (gestrichelte Funkstrecke).

6.3 Geräte-Zugriff (SSH / Telnet / SFTP / SCP / SNMP)

`NetworkObject` kapselt alle Verbindungen. Ein Login vom Typ SSH/SFTP deckt beide ab (`Login.Covers()`). `EnsureConnected()` öffnet die passende Sitzung automatisch, sobald ein Kabel vom eigenen PC zum Gerät existiert und ein passender Login gespeichert ist. Die SSH-Shell läuft als Stream mit einfacher Syntaxhervorhebung. SNMP-Aufrufe laufen ****außerhalb des UI-Threads****, damit das Fenster nicht einfriert.

6.4 Backend-Anbindung und Rechte

Nach dem Login authentifiziert `DatabaseConnection` per OAuth2 und erhält ein JWT, das **automatisch erneuert** wird (proaktiv vor Ablauf, reaktiv bei 401). Alle Änderungen (Geräte, Interfaces, Verbindungen, Logins, Rechte) werden persistiert. Das Rechte-Modell (Hidden/See/Edit/Admin/Owner) wird im Client gespiegelt und vom Backend durchgesetzt; globale Admins besitzen implizit `Owner` auf allem.

6.5 Theme, Icons und Bedienkomfort

Ein durchgängiges braunes „Ratten“-Theme (Light/Dark) mit selbst gezeichneten Vektor-Icons.

Fehler-/Infomeldungen erscheinen als themenkonformer `RatDialog` (statt System-`MessageBox`).

App-weiter Zoom (50–300 %), Erststart-Setup (Desktop-Verknüpfung + optionale nmap-Installation) und ein „Open log folder“-Button in den Einstellungen runden die Bedienung ab. Das Hauptfenster startet **maximiert**.

6.6 Funktionsübersicht für Anwender und Administratoren

Jeder angemeldete Benutzer kann:

- Geräte (PC, Router, Switch, Server, Client, Hub, Access Point, Internet-Cloud) per

Drag & Drop auf die Leinwand ziehen und frei platzieren.

- Geräte mit **Kabeln** verbinden (Connection-Tool); ein Kabel per **Doppelklick** oder

Rechtsklick bearbeiten (Name, Typ kabelgebunden/WLAN, Geschwindigkeit, Notiz) und mit dem

Delete-Tool löschen. WLAN-Strecken werden **gestrichelt** dargestellt.

- Die Leinwand **verschieben** (Drag-to-Pan / Scrollbalken) und die UI **zoomen (50–300 %)**.
- Pro Gerät eine **SSH-/Telnet-Shell** öffnen und Dateien per **SFTP/SCP** übertragen

(automatischer Verbindungsaufbau bei vorhandenem Kabel + Login).

- **SNMP** Get/Set/Walk ausführen und den **MIB-Browser** nutzen.
- Das lokale Netz per „**Discover devices**“ (nmap + tracer) automatisch erkennen.
- Offene Ports anzeigen („**Show ports**“) und Interface-Labels einblenden

(„**Show interfaces**“).

- **Interfaces** und **Logins** pro Gerät verwalten.
- Zwischen **Dark/Light-Mode** wechseln und das **eigene Konto** (Name/Passwort) ändern.
- Den **Log-Ordner öffnen** (Einstellungen → Diagnostics).
- Ohne Backend im **Offline-Modus** arbeiten („Use locally only“).

Zusätzlich kann ein Administrator (globaler Admin, `is_admin`):

- Über „**👤 Users**“ Benutzer **anlegen, bearbeiten** (Name, Passwort, Admin-Flag, „CanCreate“)

und **löschen**.

- Im **Access-Control-Tab** eines Geräts pro Benutzer eine Rechte-Stufe vergeben

(**Hidden / See / Edit / Admin / Owner**).

- Besitzt implizit **Owner-Rechte auf jedem Gerät** und darf alles löschen.

6.7 Installation, Start und Auslieferung

Auslieferung (bin/): Mit `dotnet publish ... -r win-x64 --self-contained true -o bin` wird

eine **eigenständige** Windows-Anwendung erzeugt — das kompilierte Programm (`RAT_WPF.exe`)

mit allen Abhängigkeiten (DLLs wie `Renci.SshNet.dll`, `SharpSnmpLib.dll`,

`System.Management.dll`, die .NET-Laufzeit sowie die als Ressourcen eingebetteten Bilder/Icons).

Es muss kein .NET vorinstalliert sein. Die `RAT_WPF.exe` trägt das **RAT-Logo als Icon**

(eingebettet über `<ApplicationIcon>Assets\logo.ico</ApplicationIcon>`).

Start: `RAT_WPF.exe` ausführen. Beim **Erststart** bietet RAT an, eine

Desktop-Verknüpfung (mit RAT-Icon, über `SetupService.CreateDesktopShortcut`) anzulegen und

nmap zu installieren. Danach **anmelden** (Server + Benutzer) oder „**Use locally only**“.

Aus dem Quellcode: `dotnet run --project src/RATClient/RAT_WPF/RAT_WPF.csproj`

(benötigt das .NET 9 SDK).

7. Mögliche Probleme und ihre Lösung

Problem	Ursache	Lösung
<code>NetworkObject</code> war gleichzeitig Klasse und Namespace	Namensgleichheit	Namespace in <code>NetworkObject_UI</code> umbenannt.
<code>PasswordBox</code> lässt sich nicht an eine Property binden	WPF-Sicherheitsdesign	Workaround für die Passwort-Bindung in <code>LoginView</code> .
Name-Änderung eines Geräts schlug nicht auf die UI durch	fehlende <code>PropertyChanged</code> -Meldung	ViewModel meldet Namensänderung korrekt.
Löschen eines Knotens ließ „verwaiste“ Verbindungen zurück	Verbindung im Interface des Gegenübers blieb gesetzt	Beim Löschen werden beide Interface-Verbindungen entfernt.
Geräte wurden trotz Backend-Fehler (500) angezeigt	UI fügte vor dem Speichern hinzu	Erst speichern, dann zeichnen.
Drag & Drop startete nicht auf dem Icon	transparente Icon-Pixel ließen Klicks „durchfallen“	gesamter Knoten ist hit-testbar (<code>Background=Transparent</code>).
Admin ohne Rechtezeile ließ den Login abstürzen	fehlende Berechtigungszeile	implizite, „lazy“ vergebene Owner-Zeile für globale Admins.
JWT lief nach 30 min ab → Client „starb“	Token-Ablauf	automatische, single-flight JWT-Erneuerung (proaktiv + reaktiv bei 401).
Endlosschleife „add a connection first“ sperrte das Settings-Fenster	<code>SelectionChanged</code> der äußeren Tabs feuerte den Shell-Handler erneut	Guard auf <code>OriginalSource</code> + Re-Entrancy-Flag.
Discovery dauerte sehr lange	voller <code>-F -O</code> -Scan des ganzen Subnetzes vorab	erst schneller Ping-Sweep + Topologie, dann Port-/OS-Scan pro Gerät im Hintergrund.
Ports erschienen nie am Knoten	langsamer <code>-O</code> -Scan blockierte die Port-Anzeige	Ports werden direkt nach dem schnellen <code>-F</code> -Pass angezeigt; <code>-O</code> läuft entkoppelt danach und kann den Prozess bei Überlauf beenden.
Kabel ließen sich nicht anklicken/löschen	transparenter Geräte-Canvas überdeckte die Kabel	Layer-Reihenfolge gedreht (Verbindungen oben, <code>{x:Null}</code> - Hintergrund).
SNMP traf das falsche Interface / NRE	Vergleich nur über die Subnetzmaske + null- IP	Vergleich der echten Netzadresse (<code>ip & mask</code>) mit sicherem Fallback.
Cloud lag außerhalb der Leinwand	negative Y-Position bei frischer Discovery	Position wird auf die Leinwand geklemmt.

8. Projekttagbuch

Wer hat wann an was gearbeitet (ohne DBI-Teil). Einträge ab 18.06. ergänzt aus den
Commit-Nachrichten; Aufgaben mit KI-Unterstützung sind entsprechend markiert.

21.05.2026

- **Christof:** Initial Commit; Basis-.NET-Struktur; Basis mancher Logik-Layer-Klassen (noch nicht komplett implementiert).
- **Beide:** Projektplanung und UML.

27.05.2026

- **Tobias:** Basisstruktur der Logik-Layer-Klassen (Account/User-Defaultstruktur, Basis-Implementierung von Login, NetworkObject, NetworkObjectSettings, SnmpSettings).

28.05.2026

- **Tobias:** Basisstruktur von Account und User.
- **Christof:** Basisstruktur von `NetworkObjectInterface`; Beginn der Implementierung von `IDatabaseConnection` (Funktionsumfang vorgegeben).

30.05.2026

- **Christof:** Implementierung von `NetworkObject`; SSH-Verbindung + asynchrone Befehlsausführung, SFTP- und SCP-Up-/Download; SNMP-Get/Set bei `NetworkObject`.

31.05.2026

- **Christof:** Beginn der `DatabaseMock`-Implementierung; Validierungen für Port und IP.
- **Tobias:** MVVM-Basisstruktur; `LoginView` / `LoginViewModel` (noch ohne API); Grundstruktur von `TopologyView` / `TopologyViewModel` (Hauptfenster mit Canvas).

02.06.2026

- **Christof:** `NetworkObjectSettingsWindow`, `LoginControl`, `UpdateLoginWindow`, `EnterPasswordWindow`; eigene Geräteinfos (GPU/RAM/CPU/Name) und Interfaces; `ListDirectory` über SFTP; Verbesserung des Code-Layouts.

03.06.2026

- **Christof:** `UpdateInterfaceWindow` zum Hinzufügen/Ändern von Interfaces; Änderungen in `NetworkObjectSettingsWindow`; SSH-Bugfixes; SSH-Shell-Streams.
- **Tobias:** Drag-and-Drop von der Seitenleiste in den Canvas; Bugfix (Namespace `NetworkObject` → `NetworkObject_UI`); Verknüpfung von NetworkObject und Settings-Fenster.

10.06.2026

- **Christof** (mit Unterstützung von Claude): Debug-Modus (überspringt `LoginView`); Bugfixes.
- **Tobias:** Restrukturierung von `TopologyView` / `ViewModel` (Canvas als reines `Canvas`-Element, weniger komplex); NetworkObjects werden per Binding aus einer Liste gezeichnet und beim Drop hinzugefügt; Speicherung der Position (x, y).

11.06.2026

- **Christof** (mit Unterstützung von Claude): Light/Dark-Mode; RAT-Icons; `SettingsWindow`; MIB-Browser; Permissions; Löschen von NetworkObjects; `SelectInterfaceWindow`; Redesign von Interface/Login; Access Control; themenkonformes SSH-Terminal mit Syntaxhervorhebung.
- **Tobias:** „Tools“ in der TopologyView (an ein Enum gebunden); Löschen von NetworkObjects über das Tool; `NetworkConnection` (Logik vorhanden, noch nicht gezeichnet).

13.06.2026

- **Tobias:** `NetworkConnectionViewModel`; Zeichnen der Verbindungen als Linien, die sich beim Bewegen der Endknoten aktualisieren (Binding an `ObservableCollection`).

14.06.2026

- **Tobias:** Bugfix — Löschen eines NetworkObject-VM entfernte die Verbindung im Interface des Gegenübers nicht (verhinderte neue Verbindungen); `SelectInterfaceWindow` zeigt nur noch freie Interfaces.

16.06.2026

- **Christof** (mit Unterstützung von Claude): Anbindung an die Datenbank (`DatabaseConnection` über HTTP statt Mock); `DatabaseConnectionStore`; echter Login gegen das Backend; Laden des gespeicherten Graphen nach dem Login; Persistieren von Anlagen/Verschieben/Löschen; CRUD für Interfaces/Verbindungen/Berechtigungen; admin-only `ManageUsersWindow`.

- **Tobias:** Bugfix beim Binding des Namens zwischen NetworkObject-VM und -View.

17.06.2026

- **Christof** (mit Unterstützung von Claude): Globale Admins erhalten Owner-Rechte auf allem; Löschen persistiert nun zuverlässig; gespeicherte Verbindungen werden nach dem Login gezeichnet; Logout-Button (merkt sich Server-IP/Port); rat-themed Login-Icons + Login-Status; Bearbeiten von Benutzern (Name/Passwort/Admin/CanCreate); About-Fenster mit Ratten-Maskottchen; `RatDialog` statt System-`MessageBox`; Export aller Vektorgrafiken als PNG; automatische JWT-Erneuerung; app-weiter Zoom (50–300 %).

18.06.2026

- **Tobias:** Basis der Dokumentation (`Dokumentation.md`) und Projekttagbuch; Fix für `PasswordBox` -Bindung im LoginView.
- **Christof** (mit Unterstützung von Claude): API-Dokumentation (Deutsch), Klassendiagramm-Howto, 10 xUnit-Tests, abstrakte `DeviceDescriptor` -Hierarchie + XML-Doku; Passworrichtlinie (Front- und Backend); Rolling-File-Logging; Zoom/ShowInterfaces aus den Backend-Einstellungen; Kabel-Interface-Labels; Telnet; vereinheitlichte SSH/SFTP-Logins; Auto-Connect; „Use locally only“-Modus; Discovery mit nmap; Erststart-Setup; nmap-basierte Topologie (Switch/Hub); „Show ports“; Geräte-Infos (Subnetzmaske/OS/Hostname); traceroute-Router + Internet-Cloud; Cisco-Interface-Benennung; Kreisanordnung; `DeviceClassifier`; Access-Point + WLAN-Modellierung; pan-bare Leinwand; SNMP-Bugfix; ausführlicheres Logging; „Open log folder“; Performance- und Anzeigekorrekturen bei Discovery/Ports/Kabeln.

9. Einsatz von KI

9.1 Vorgehensweise

- **Was wurde mit KI umgesetzt?** Große Teile des UI-Feinschliffs und der Erweiterungen

(Theming/Icons, Discovery, SNMP-/Logging-Verbesserungen, Bugfixes, Tests, Dokumentation)

wurden mit KI-Unterstützung umgesetzt. Jede KI-gestützte Änderung ist im Code mit

```
//KI start ... //KI end
```

 (bzw. „ported to MVVM by AI“) markiert und in

`AI_usage.md` prompt-weise protokolliert.

- **Ziele:** schnellere Umsetzung von Routine- und Feinschliff-Aufgaben, konsistentes Theme,

robustere Fehlerbehandlung, bessere Dokumentation.

- **Eingesetzte KI-Tools:** Claude (Agent / Claude Code), ergänzend Gemini.
- **Kosten:** im Rahmen der genutzten Abos/Token-Kontingente.

9.2 Reflexion

- **Wo war KI hilfreich?** Bei wiederkehrenden UI-/MVVM-Mustern, beim Aufspüren subtiler

WPF-Eigenheiten (Hit-Testing, Bindings), bei Tests und beim Verfassen der Dokumentation.

- **Was würden wir anders machen?** Architekturentscheidungen früher festzurren und der KI

engere, klar abgegrenzte Aufgaben geben.

- **Was lief gut / schlecht?** Gut: Tempo und konsistenter Stil. Weniger gut: gelegentlich zu

breit angelegte Änderungen, die manuelle Nachkontrolle (Tests, Reviews) erforderten.

10. Quellen für Bilder und Medien

Medium	Quelle / Urheber	Hinweis
RAT-Icons (Geräte, Status, Interfaces)	selbst erstellt mit Claude	WPF- <code>DrawingImage</code> -Vektoren, als PNG exportiert unter <code>doc/assets/vectorgraphics/</code> .

RAT-Logo	Tobias	<code>doc/assets/logo.png</code> .
About-Maskottchen	Tobias & Claude (Idee/Helfer: Christof)	<code>Icon.AboutRat</code> .
UML-Klassendiagramm	selbst erstellt	<code>doc/assets/drawio/uml.drawio(.png)</code> , <code>doc/assets/images/screenshots/uml.jpg</code> .
GUI-Sketches	selbst erstellt	<code>doc/assets/images/sketches/</code> .
Milestones/Projektzeitplan	selbst erstellt	<code>doc/assets/images/screenshots/Milestones_und_Projektzeitplan.png</code> .
Markdown-Stylesheet	< https://github.com/sumpfel/RAT-Backend > — GPL v3, Urheber: <i>Sumpfel</i>	siehe <code>Licenses.md</code> .
<code>ComparisonConverter</code> (Enum-Bindung)	Stack Overflow < https://stackoverflow.com/a/2908885 > (CC BY-SA 4.0)	im Code referenziert.